

if statement

Mr. Poole
Java

So far we've learned

- Data types
- Variables
- Output
- Input
- Math
- Random

These all help us run very linear programs.

Start to finish,
straightforward results.

What if we wanted to only
run some of our code?

Starting with the answer

booleans! booleans can ONLY be true/false values.

This means they are the answer to a Yes/No question.

True

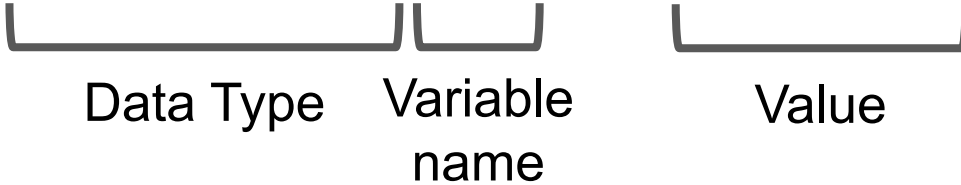


False



boolean variables

```
boolean x = true;  
boolean y = false;
```



Primitive boolean Expressions

```
==  is equal  
!=  is NOT equal  
<   less than  
>   greater than  
<=  less than or equal to  
>=  greater than or equal to
```

boolean expressions are our way to asking YES/NO questions! They always result to true/false.

Primitive boolean Expressions

integers
can be
compared



- `==` is equal
- `!=` is **NOT** equal
- `<` less than
- `>` greater than
- `<=` less than or equal to
- `>=` greater than or equal to

Primitive boolean Expressions

doubles
can be
compared



- `==` is equal
- `!=` is **NOT** equal
- `<` less than
- `>` greater than
- `<=` less than or equal to
- `>=` greater than or equal to

Primitive boolean Expressions

booleans
can be
compared

`==` is equal

`!=` is **NOT** equal

`<` less than

`>` greater than

`<=` less than or equal to

`>=` greater than or equal to

Primitive boolean Expressions

**Strings
CAN NOT
be compared**

```
==  is equal
!=  is NOT equal
<   less than
>   greater than
<=  less than or equal to
>=  greater than or equal to
```

Primitive boolean Expressions & variables

```
boolean a = 5 < 2;
```

```
boolean b = 3 != 4;
```

```
boolean c = 14 <= 3;
```

Primitive boolean Expressions & variables

```
boolean a = 5 < 2;
```

a = true;

```
boolean b = 3 != 4;
```

b = true;

```
boolean c = 14 <= 3;
```

c = false;

Using variables as boolean expressions

```
int num1 = 5;  
double num2 = 3;  
boolean d = num1 < num2;
```

Variables are
able to be used
in boolean
expressions!

Okay, now we know about how to
get a YES/NO result.

Now let's learn about how to
change what code runs based on
that answer!

if statements

An if statement is a coding structure that allows us to choose what code runs based on a condition. Think of it like a choose your own adventure!



if statements

An if statement is a multi-line command. Notice, it does NOT require a semicolon but replaces it with brackets to continue the command.

```
if( /* Condition */ ){  
  
}
```

if statements

```
if( /* Condition */ ){  
    // This code will run IF  
    // the condition above is true.  
}
```

if statement conditions

The condition of a if statement must result in a true/false answer.



```
if(true){ }
```

if statement example

```
if(true){  
    System.out.println("It worked!");  
}
```

```
if(false){  
    System.out.println("It worked!");  
}
```

if statement example with a boolean variable

```
boolean answer = x < y;  
if(answer){  
    System.out.println(x + " is smaller than y!");  
}
```

if statement - Lab

- Write a program that has 2 int variables with values.
 - Initialize them with any value you choose
- Write a message to the screen if the two values are different or if they are the same
- Verify your code works by trying different values